Machine Coding: Client-Side Pagination for Large API Data

- **Problem:** Fetch a large list once from an API (500 comments from `jsonplaceholder.typicode.com/comments`) and show it page by page on the client: First / Prev / page numbers with ellipsis / Next / Last, a "per page" selector, "Showing 21–40 of 500", loading / error / empty states, and the current page kept in the URL.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Assumptions written down
Use cases + architecture + state + the math	8 min	Table, tree, the slice formula
Fetch + loading/error + Prev/Next slicing	15 min	Working pagination
Page numbers with ellipsis, First/Last	12 min	Scales to any page count
Page size, URL sync, clamping, focus	10 min	Polished behaviour
Test manually + wrap-up	10 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"I'll store only the page number and page size. Total pages, the slice and the 'Showing X–Y' text are all derived, so they can never disagree."* If you run out of time, a clear plan you talked through still scores well.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Fetch all items once; show a loading state, then the first page.
- Show `pageSize` items per page (10 / 20 / 50, selectable).
- Controls: First, Prev, numbered pages with "…" for gaps, Next, Last. Disable what doesn't apply (Prev on page 1, Next on the last page).
- Show "Showing 21–40 of 500".
- Error state with Retry; empty state when the API returns no items.
- Keep `page` and `size` in the URL so a refresh or a shared link opens the same page.

Non-functional requirements

Concern	What it means here
Performance	500 items in memory is trivial; `slice` is O(pageSize) per render. Render only the current page, never all 500 cards.
Scalability	The page bar shows at most 7 numbers whether there are 5 pages or 5,000. Past ~10k items, switch to server-side pagination (same UI, different data source).
Error handling	Check `response.ok` (fetch doesn't throw on 404/500); validate the response shape; abort on unmount; clamp invalid pages from the URL.
Accessibility	`<nav aria-label="Pagination">`, `aria-current="page"` on the active button, `aria-label`s on icon buttons, `aria-live` on "Showing…", focus moves to the heading after a page change.
Reusability	`Pagination` is stateless (`currentPage`, `totalPages`, `onPageChange`) and works with any data. The math lives in pure functions.
Extensibility	Server-side pagination later: keep `Pagination` and pass the total count from the API instead of `items.length`.

Clarifying questions to ask the interviewer

1. How big is the data: hundreds, thousands, millions? Is fetching everything at once OK (client-side), or must the server paginate?
2. What page sizes should we offer, and what is the default?
3. What should the controls look like: just Prev/Next, or numbered pages? With ellipsis? First/Last?
4. Should the page survive a refresh / be shareable by URL? Should the browser Back button go to the previous page?
5. When the page size changes, should we jump to page 1 or keep the user's place?
6. Do we need search/sort together with pagination? (If yes: changing them resets to page 1.)

Assumptions we will build with

- Client-side: one request for all 500 items, then `slice`.
- Page sizes 10 / 20 / 50, default 10.

- Numbered pages with ellipsis, always 7 slots (1 … 4 5 6 … 50), plus First/Prev/Next/Last.
- Changing the page size keeps the first visible item on screen.
- The URL holds `?page=&size=` via `replaceState` (no history entry per click).
- Invalid URL values fall back to defaults; a page past the end is clamped to the last page.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Open the page	"Loading comments..." then page 1, "Showing 1–10 of 500"
API returns 404/500 or the network fails	Error message + Retry; Retry shows loading again and refetches
API returns `[]`	"No comments found."; no pagination bar
Click Next / Prev	Next/previous page; Prev disabled on page 1, Next disabled on the last page
Click First / Last (« / »)	Jump to page 1 / the last page
Click page "5"	Page 5 shown; button highlighted with `aria-current="page"`
Many pages (50)	Bar shows `1 2 3 4 5 … 50`, `1 … 4 5 6 … 50`, or `1 … 46 47 48 49 50`
Few pages (≤ 7)	All numbers, no ellipsis
Only one page	No pagination bar
On page 3 (size 10, items 21–30), change size to 20	Page 2 (items 21–40): the first item stays visible
Change page	Focus moves to the heading (screen readers + scroll to top); "Showing…" announced
Any page change	URL becomes `?page=N&size=S`
Open `?page=999&size=20`	Clamped to the last page (25); URL fixed to `?page=25&size=20`
Open `?page=abc&size=7`	Falls back to page 1, size 10
Last page not full (e.g. 501 items)	"Showing 501–501 of 501" with 1 card

3. Design the architecture

Module	Responsibility
`App`	Owns the data + request status + `{ page, pageSize }`; derives everything else; syncs the URL
`services/commentsService.js`	`fetchComments({ signal })`: fetch, `response.ok` check, shape check
`utils/pagination.js`	Pure math: `getTotalPages`, `paginate`, `getPageRange`, `pageForNewSize`, `clamp`, `readPaginationFromURL`
`Pagination`	Stateless page bar (First/Prev/numbers/…/Next/Last)

Module	Responsibility
`CommentCard`	Shows one comment

```
App  (comments, status, error, attempt, { page, pageSize })
├── status message      (loading / error + Retry / empty)
├── toolbar             "Showing X–Y of N" + per-page <select>
├── <ul> visible.map
│     └── CommentCard    ← comment
└── Pagination          ← currentPage, totalPages → onPageChange(n)
utils/pagination.js     ← pure math used by App and Pagination
services/commentsService.js ← fetch used by App
```

Start simple. First version: everything in `App` with Prev/Next only (the 7-line formula). Extract `Pagination` when you add numbered pages; move the math to `utils` once there are 3+ helpers.

4. Design the state / data model

State	Purpose
`comments`	All items from the API (the full list, never mutated)
`status`	`loading | success | error`, one enum
`error`	Message for the error state
`attempt`	Retry counter; changing it re-runs the fetch effect
`{ page, pageSize }`	The only pagination state (one object, so a page-size change updates both at once)

```
const [comments, setComments] = useState([]);
const [status, setStatus] = useState("loading"); // loading | success | error
const [error, setError] = useState("");
const [attempt, setAttempt] = useState(0);
const [{ page, pageSize }, setPagination] = useState(() =>
readPaginationFromURL(window.location.search));

// derived
const totalPages = getTotalPages(comments.length, pageSize); // Math.max(1, Math.ceil(n / size))
const currentPage = clamp(page, 1, totalPages);
const visible = paginate(comments, currentPage, pageSize);   // slice(start, start + size)
const firstItem = comments.length === 0 ? 0 : (currentPage - 1) * pageSize + 1;
const lastItem = Math.min(currentPage * pageSize, comments.length);
```

The core formula (say this out loud): `totalPages = ceil(total / size)`, `start = (page − 1) × size`, `end = start + size`, `visible = items.slice(start, end)`.

Senior note

- **Derived, not stored:** `totalPages`, `currentPage` (clamped), `visible`, `firstItem`, `lastItem`, and the list of page buttons. Storing `currentData` in state means re-slicing in every handler, and it goes stale when the page size changes.
- **Store the raw `page`, derive the safe `currentPage`:** the stored page can be out of range (from the URL, or before the data loads); clamping at render time makes it impossible to show an empty page 999.
- **Ownership:** `App` owns the data and the position; `Pagination` only reports clicks. The URL mirrors `{ page, size }` (read once, written after a successful load); it is not a second source of truth.

5. Design the data structures and contracts

Comment (from the API, stable numeric `id` used as the React key):

```
{ "postId": 1, "id": 1, "name": "id labore ex et quam laborum", "email": "Eliseo@gardner.biz", "body": "laudantium enim quasi est quidem..." }
```

API contract

Call	Input	Success	Failure
`fetchComments({ signal })` ≈ `GET /comments`	`AbortSignal`	`Comment[]` (500 items)	throws `Error("Could not load comments (HTTP 404).")`, or `AbortError` when cancelled (ignored)

Server-side version (when data is huge): `GET /comments?_page=3&_limit=20` returning the items, plus the total in an `X-Total-Count` header. Only the data source changes; `Pagination` stays the same.

Component contract: `<Pagination currentPage={3} totalPages={50} onPageChange={(n) => …} />`.

Page range contract: `getPageRange(current, total, siblings = 1)` returns numbers and ellipsis markers, always 7 slots when `total > 7`:

```
getPageRange(1, 50)  // [1, 2, 3, 4, 5, "end-ellipsis", 50]
getPageRange(5, 50)  // [1, "start-ellipsis", 4, 5, 6, "end-ellipsis", 50]
getPageRange(48, 50) // [1, "start-ellipsis", 46, 47, 48, 49, 50]
```

The ellipsis markers are distinct strings so they can be used as React keys.

6. Trace the data flow

User on page 3 (size 10) picks "20 per page":

```
User selects 20 in the per-page <select>
  ↓ changePageSize: first visible item = (3 − 1) × 10 + 1 = 21
  ↓ setPagination({ page: floor(20 / 20) + 1 = 2, pageSize: 20 })
  ↓ React re-renders App
  ↓ derived: totalPages = ceil(500 / 20) = 25, currentPage = 2, visible = slice(20, 40)
```

```
  ↓ effect: history.replaceState(…, "?page=2&size=20")
  ↓ UI: "Showing 21–40 of 500", bar 1 [2] 3 4 5 … 25. Item 21 is still on screen
```

7. Important technical challenges

Challenge 1: off-by-one and out-of-range pages

- *Problem:* `startIndex = currentPage * size` skips the first page. With empty data `totalPages = 0`, so `currentPage === totalPages` is false and Next stays enabled (page 2 of 0). A shared link `?page=999` shows an empty page.
- *Solution:* One tested formula, `totalPages` is at least 1, and the page is clamped when deriving.

```
export const getTotalPages = (total, size) => Math.max(1, Math.ceil(total / size));
export const paginate = (items, page, size) => items.slice((page - 1) * size, (page - 1) * size + size);
const currentPage = clamp(page, 1, totalPages);
```

Challenge 2: page buttons that scale (ellipsis)

- *Problem:* `Array.from({ length: totalPage })` renders 500 buttons for 500 pages. A naive ellipsis also makes the bar change width on every click, and can hide a single page behind "…" (silly: `1 … 3`).
- *Solution:* Three cases with a fixed slot count; an ellipsis always hides at least 2 pages.

```
const block = siblings * 2 + 3;                          // 5 when siblings = 1
if (total <= block + 2) return all pages;                 // ≤ 7: no ellipsis
if (current <= siblings + 3) return [1..block, "end-ellipsis", total];
if (current >= total - siblings - 2) return [1, "start-ellipsis", last block];
return [1, "start-ellipsis", current-1, current, current+1, "end-ellipsis", total];
```

Challenge 3: fetching safely

- *Problem:* `fetch` resolves on HTTP 404/500, so `response.json()` gives an error object, and `.slice` on it crashes. If the component unmounts mid-request, the late response tries to update state. Faker data created *inside* the component body is regenerated on every render.
- *Solution:* Check `response.ok` and the response shape, cancel with `AbortController`, ignore `AbortError`, and load the data once in an effect.

```
if (!response.ok) throw new Error(`Could not load comments (HTTP ${response.status}).`);
if (!Array.isArray(data)) throw new Error("Unexpected response from the server.");

useEffect(() => {
  const controller = new AbortController();
  fetchComments({ signal: controller.signal }).then(…).catch((err) => { if (err.name !== "AbortError") …
});
  return () => controller.abort();
}, [attempt]);
```

Challenge 4: keeping the user's place (page size + URL + focus)

- *Problem:* On page 30 of size 10 (items 291–300), switching to size 50 and resetting to page 1 loses the user's place. After clicking "Next" at the bottom of a long page, the user stays scrolled down and a screen reader hears nothing. A refresh goes back to page 1.
- *Solution:* Recompute the page from the first visible item, move focus to the heading, and mirror the position into the URL (only after the data loads, so the clamp uses the real total).

```
export const pageForNewSize = (page, oldSize, newSize) => Math.floor(((page - 1) * oldSize) / newSize) +
1;

const goToPage = (next) => {
  setPagination((prev) => ({ ...prev, page: clamp(next, 1, totalPages) }));
  headingRef.current?.focus(); // <h1 tabIndex={-1}>
};

useEffect(() => {
  if (status !== "success") return;
  window.history.replaceState(null, "", `?page=${currentPage}&size=${pageSize}`);
}, [status, currentPage, pageSize]);
```

8. Implement in the correct order

#	Step	Working result
1	Fetch in `useEffect`, render all items with `key={item.id}`	Data on screen
2	`page` state + the slice formula + Prev/Next (with disabled states)	Basic pagination
3	`status` enum: loading, error (with `response.ok` check), Retry	Async states
4	`getTotalPages` ≥ 1 + clamp → empty data and edge pages safe	No off-by-one
5	"Showing X–Y of N"	Clear position
6	Extract stateless `Pagination`; numbered buttons + `aria-current`	Direct page jumps
7	`getPageRange` with ellipsis + First/Last	Scales to any page count
8	Page-size select + `pageForNewSize`	Keeps the user's place
9	`AbortController` cleanup	No late updates
10	URL read (lazy init) + write (`replaceState`) + validation	Refresh/share works
11	Focus the heading on page change, `aria-live`, responsive grid	Accessible + polished
12	Self-check `getPageRange` for every (current, total) up to 60	Math proven

9. Cover all states and edge cases

State / edge case	Handling
Initial / loading	"Loading comments..." (`role="status"`)
Success	Current page of cards + toolbar + pagination bar
Error (HTTP or network)	Message with the HTTP status + Retry (`role="alert"`)
Empty data	"No comments found."; no toolbar, no bar
Exactly one page	Bar hidden
Last page partly filled	Correct "Showing 491–500"; fewer cards
Page 1 / last page	Prev+First / Next+Last disabled
≤ 7 pages	All page numbers, no ellipsis
Many pages	7 slots; an ellipsis always hides ≥ 2 pages
Page size change	First visible item stays on screen
`?page=999`	Clamped to the last page; URL corrected after load
`?page=abc` , `?page=-3` , `?size=7`	Fall back to page 1 / size 10
Unmount or Retry during a request	Request aborted; `AbortError` ignored
Clicking the current page	Same page (no-op re-render)
Narrow screen	Bar wraps; grid becomes 1 column

10. Final structure

```
_04_pagination_Large_ApiData_Client_side/
├── index.html
└── src/
    ├── main.jsx
    ├── index.css                ← global reset
    ├── App.jsx                  ← state owner: data, status, { page, pageSize }; derived slice; URL
sync
    ├── App.css                  ← grid, cards, pagination bar
    ├── services/
    │   └── commentsService.js   ← fetchComments (ok + shape checks, abortable)
    ├── utils/
    │   └── pagination.js        ← getTotalPages, paginate, getPageRange, pageForNewSize, clamp, URL
parse
    └── components/
        ├── Pagination.jsx       ← stateless page bar
        └── CommentCard.jsx      ← one item
```

11. Full working code

`src/services/commentsService.js`

```
const API_URL = "https://jsonplaceholder.typicode.com/comments"; // 500 items

// GET /comments -> [{ postId, id, name, email, body }]
export async function fetchComments({ signal, fail = false } = {}) {
  const response = await fetch(fail ? `${API_URL}-broken` : API_URL, { signal });
  if (!response.ok) throw new Error(`Could not load comments (HTTP ${response.status}).`); // fetch does
not throw on 404/500
  const data = await response.json();
  if (!Array.isArray(data)) throw new Error("Unexpected response from the server.");
  return data;
}
```

`src/utils/pagination.js`

```
export const PAGE_SIZES = [10, 20, 50];
export const DEFAULT_PAGE_SIZE = 10;

export const clamp = (value, min, max) => Math.min(Math.max(value, min), max);

export function getTotalPages(totalItems, pageSize) {
  return Math.max(1, Math.ceil(totalItems / pageSize)); // never 0, so "page 1 of 1" holds for empty
data
}

export function paginate(items, page, pageSize) {
  const start = (page - 1) * pageSize;
  return items.slice(start, start + pageSize);
}

// Page buttons with ellipsis, always the same count so the bar does not jump:
// (5, 50) -> [1, "start-ellipsis", 4, 5, 6, "end-ellipsis", 50]
export function getPageRange(current, total, siblings = 1) {
  const block = siblings * 2 + 3; // current + siblings + one neighbour of first/last
  if (total <= block + 2) return Array.from({ length: total }, (_, i) => i + 1);

  if (current <= siblings + 3) {
    return [...Array.from({ length: block }, (_, i) => i + 1), "end-ellipsis", total];
  }
  if (current >= total - siblings - 2) {
    return [1, "start-ellipsis", ...Array.from({ length: block }, (_, i) => total - block + 1 + i)];
  }
  const middle = Array.from({ length: siblings * 2 + 1 }, (_, i) => current - siblings + i);
  return [1, "start-ellipsis", ...middle, "end-ellipsis", total];
```

```
}

// Keep the first visible item on screen when the page size changes.
export function pageForNewSize(page, oldSize, newSize) {
  return Math.floor(((page - 1) * oldSize) / newSize) + 1;
}

// Untrusted URL values -> safe numbers. The page is clamped later, once the data length is known.
export function readPaginationFromURL(search) {
  const params = new URLSearchParams(search);
  const page = parseInt(params.get("page"), 10);
  const size = parseInt(params.get("size"), 10);
  return {
    page: Number.isInteger(page) && page > 0 ? page : 1,
    pageSize: PAGE_SIZES.includes(size) ? size : DEFAULT_PAGE_SIZE,
  };
}
```

src/App.jsx

```
import { useEffect, useRef, useState } from "react";
import "./App.css";
import { fetchComments } from "./services/commentsService";
import {
  PAGE_SIZES,
  clamp,
  getTotalPages,
  pageForNewSize,
  paginate,
  readPaginationFromURL,
} from "./utils/pagination";
import Pagination from "./components/Pagination";
import CommentCard from "./components/CommentCard";

const FAIL_DEMO = new URLSearchParams(window.location.search).has("fail"); // add ?fail to see the error
state

function App() {
  const [comments, setComments] = useState([]);
  const [status, setStatus] = useState("loading"); // loading | success | error
  const [error, setError] = useState("");
  const [attempt, setAttempt] = useState(0);
  const [{ page, pageSize }, setPagination] = useState(() =>
readPaginationFromURL(window.location.search));
  const headingRef = useRef(null);

  useEffect(() => {
```

```
  const controller = new AbortController(); // cancel the request on unmount / retry
  fetchComments({ signal: controller.signal, fail: FAIL_DEMO && attempt === 0 })
    .then((data) => {
      setComments(data);
      setStatus("success");
    })
    .catch((err) => {
      if (err.name === "AbortError") return;
      setError(err.message);
      setStatus("error");
    });
  return () => controller.abort();
}, [attempt]);

// Derived: clamp the page against the real data, so ?page=999 shows the last page.
const totalPages = getTotalPages(comments.length, pageSize);
const currentPage = clamp(page, 1, totalPages);
const visible = paginate(comments, currentPage, pageSize);
const firstItem = comments.length === 0 ? 0 : (currentPage - 1) * pageSize + 1;
const lastItem = Math.min(currentPage * pageSize, comments.length);

useEffect(() => {
  if (status !== "success") return; // don't overwrite the URL page before we know the real total
  window.history.replaceState(null, "", `?page=${currentPage}&size=${pageSize}`);
}, [status, currentPage, pageSize]);

const goToPage = (next) => {
  setPagination((prev) => ({ ...prev, page: clamp(next, 1, totalPages) }));
  headingRef.current?.focus(); // move screen readers + scroll to the top of the new page
};

const changePageSize = (e) => {
  const newSize = Number(e.target.value);
  setPagination({ page: pageForNewSize(currentPage, pageSize, newSize), pageSize: newSize });
};

const retry = () => {
  setStatus("loading");
  setAttempt((a) => a + 1);
};

return (
  <main className="container">
    <h1 ref={headingRef} tabIndex={-1}>
      Comments
    </h1>
```

```
      {status === "loading" && <p className="message" role="status">Loading comments...</p>}

      {status === "error" && (
        <div className="message" role="alert">
          <p>{error}</p>
          <button type="button" onClick={retry}>
            Retry
          </button>
        </div>
      )}

      {status === "success" && comments.length === 0 && <p className="message">No comments found.</p>}

      {status === "success" && comments.length > 0 && (
        <>
          <div className="toolbar">
            <p aria-live="polite">
              Showing {firstItem}–{lastItem} of {comments.length}
            </p>
            <label>
              Per page{" "}
              <select value={pageSize} onChange={changePageSize}>
                {PAGE_SIZES.map((size) => (
                  <option key={size} value={size}>
                    {size}
                  </option>
                ))}
              </select>
            </label>
          </div>

          <ul className="list">
            {visible.map((c) => (
              <CommentCard key={c.id} comment={c} />
            ))}
          </ul>

          <Pagination currentPage={currentPage} totalPages={totalPages} onPageChange={goToPage} />
        </>
      )}
    </main>
  );
}

export default App;
```

src/components/Pagination.jsx

```
import { getPageRange } from "../utils/pagination";

// Stateless: knows nothing about the data, only the current page and the total.
const Pagination = ({ currentPage, totalPages, onPageChange }) => {
  if (totalPages <= 1) return null;

  const isFirst = currentPage === 1;
  const isLast = currentPage === totalPages;

  return (
    <nav className="pagination" aria-label="Pagination">
      <button type="button" onClick={() => onPageChange(1)} disabled={isFirst} aria-label="First page">
        «
      </button>
      <button type="button" onClick={() => onPageChange(currentPage - 1)} disabled={isFirst} aria-
label="Previous page">
        ‹ Prev
      </button>

      {getPageRange(currentPage, totalPages).map((item) =>
        typeof item === "string" ? (
          <span key={item} className="ellipsis" aria-hidden="true">
            …
          </span>
        ) : (
          <button
            key={item}
            type="button"
            className={item === currentPage ? "page active" : "page"}
            onClick={() => onPageChange(item)}
            aria-current={item === currentPage ? "page" : undefined}
            aria-label={`Page ${item}`}
          >
            {item}
          </button>
        )
      )}

      <button type="button" onClick={() => onPageChange(currentPage + 1)} disabled={isLast} aria-
label="Next page">
        Next ›
      </button>
      <button type="button" onClick={() => onPageChange(totalPages)} disabled={isLast} aria-label="Last
page">
        »
      </button>
    </nav>
```

```
  );
};

export default Pagination;
```

src/components/CommentCard.jsx

```
const CommentCard = ({ comment }) => (
  <li className="card">
    <div className="card-header">
      <span>#{comment.id}</span>
      <span>Post {comment.postId}</span>
    </div>
    <h3>{comment.name}</h3>
    <p className="email">{comment.email}</p>
    <p className="body">{comment.body}</p>
  </li>
);

export default CommentCard;
```

src/App.css

```
.container { max-width: 900px; margin: 0 auto; padding: 16px; font-family: system-ui, sans-serif; }
h1 { text-align: center; margin-bottom: 16px; }
h1:focus { outline: none; }

.toolbar { display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap:
8px; margin-bottom: 12px; color: #4b5563; }
select { padding: 4px 8px; border-radius: 6px; border: 1px solid #ccc; }

.list { list-style: none; display: grid; grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
gap: 12px; }
.card { border: 1px solid #ddd; border-radius: 8px; padding: 14px; background: #f9fafb; }
.card-header { display: flex; justify-content: space-between; font-size: 13px; color: #6b7280; margin-
bottom: 6px; }
.card h3 { font-size: 16px; margin-bottom: 4px; text-transform: capitalize; }
.card .email { font-size: 13px; color: #4f46e5; margin-bottom: 6px; word-break: break-all; }
.card .body { font-size: 14px; color: #4b5563; line-height: 1.5; }

.pagination { display: flex; flex-wrap: wrap; justify-content: center; align-items: center; gap: 4px;
margin: 20px 0; }
.pagination button { min-width: 40px; padding: 6px 10px; border: 1px solid #d1d5db; border-radius: 6px;
background: white; color: #111827; cursor: pointer; }
.pagination button:hover:not(:disabled) { background: #eef2ff; }
.pagination button:disabled { opacity: 0.4; cursor: not-allowed; }
```

```
.pagination .active { background: #4f46e5; border-color: #4f46e5; color: white; }
.pagination .active:hover { background: #4f46e5; }
.ellipsis { padding: 0 4px; color: #6b7280; }

.message { text-align: center; padding: 40px; color: #4b5563; display: flex; flex-direction: column;
align-items: center; gap: 12px; }
.message button { padding: 8px 16px; border: none; border-radius: 6px; background: #4f46e5; color:
white; cursor: pointer; }
```

`src/index.css`

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  background-color: white;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: off-by-one in the slice, empty data (no "page 1 of 0"), a partly filled last page, out-of-range/garbage URL values, fixed-width page bar with meaningful ellipsis, keeping the user's place on page-size change, HTTP errors that `fetch` doesn't throw, bad response shape, request cancellation, focus and scroll after a page change.

Trade-offs to mention to the interviewer

- *Client-side vs server-side pagination:* client-side means one request and instant page changes, but you download everything up front. That's fine for hundreds or a few thousand items; beyond that use `?page=&limit=` (or cursor-based pagination for feeds that change often).

- *Numbered pages vs infinite scroll:* pages give a stable position, shareable links and a reachable footer; infinite scroll suits feeds.
- `replaceState` *vs* `pushState`*:* the Back button leaves the page instead of going back one page. For Back support, `pushState` + a `popstate` listener (or React Router `useSearchParams`).
- *Keeping the place on page-size change vs resetting to page 1:* better UX; resetting to page 1 is the right choice when *filters or sort* change instead.
- *Hand-rolled fetch vs React Query / SWR:* fine for one request; a library adds caching, deduping and retries.

What to add with more time

- Tests: unit tests for `getPageRange`, `paginate`, `pageForNewSize` (the self-check above as Vitest); React Testing Library for Next/Prev disabled states, page size change and the error → Retry flow.
- Server-side pagination with `X-Total-Count`, prefetch of the next page, and a cache of visited pages.
- Search/sort combined with pagination (reset to page 1 on change).
- `pushState` + `popstate` so Back/Forward move between pages; a "Go to page" input; keyboard shortcuts (← / →).
- Skeleton cards while loading; a virtualised list if the page size grows to hundreds.

13. Counter questions the interviewer may ask

Requirements and design

Q1. When is client-side pagination the wrong choice?

When the full data is large (tens of thousands of rows, or MBs of JSON), slow to fetch, or changes often. Then the server must paginate (`?page=&limit=` or a cursor), and the client only fetches one page at a time.

Q2. Offset pagination vs cursor pagination?

Offset (`page=5&limit=20`) is simple and allows jumping to any page, but items shift if rows are added or deleted while you browse, and large offsets are slow in databases. Cursor (`after=<lastId>`) is stable and fast for feeds, but you can't jump to page 37.

Q3. Numbered pages or infinite scroll?

Pages: a stable position, shareable links, a reachable footer, better for tables and search results. Infinite scroll: better for feeds and discovery, but needs care for the Back button, the footer and memory (virtualisation).

State design

Q4. What is the minimum state for pagination?

`page` and `pageSize`. `totalPages`, the slice, "Showing X–Y" and the page buttons are all derived. Storing `currentData` means re-slicing in every handler, and it goes stale on page-size changes.

Q5. Why store the raw `page` but render a clamped `currentPage`?

The stored page can be invalid (from the URL, or before the data arrives). Clamping at render time (`clamp(page, 1, totalPages)`) means an out-of-range page can never be shown, and we don't need an effect to "fix" the state.

Q6. Why are `page` and `pageSize` in one state object?

Changing the page size also changes the page (to keep the user's place). One `setPagination({ page, pageSize })` updates both at once, with no in-between render where they disagree.

Hard parts

Q7. Write the formula for the current slice.

`totalPages = Math.max(1, Math.ceil(total / size))`, `start = (page - 1) * size`, `visible = items.slice(start, start + size)`. `slice` handles a partly filled last page, and `Math.max(1, …)` avoids "page 1 of 0".

Q8. Why did "Next" stay enabled with empty data in the first version?

`totalPages = Math.ceil(0 / 10) = 0`, and the check was `currentPage === totalPage` → `1 === 0` → false, so Next stayed enabled and you could go to page 2 of 0. Clamping `totalPages` to at least 1 fixes it.

Q9. How does your ellipsis algorithm work?

There are always 7 slots when there are more than 7 pages: first, last, the current page ± 1, and an ellipsis on each side. Near the start it shows `1 2 3 4 5 … 50`; near the end `1 … 46 47 48 49 50`; otherwise `1 … 4 5 6 … 50`. A fixed slot count stops the bar from jumping, and an ellipsis always hides at least 2 pages (never `1 … 3`).

Q10. Page size changes from 10 to 50 while on page 30. Which page do you show?

The page that contains the first item currently visible: `floor(((30 - 1) * 10) / 50) + 1 = 6`. Item 291 stays on screen. Reset to page 1 only when *filters or sort* change.

Q11. Why must the ellipsis keys be distinct?

Both ellipses are rendered in the same list; two `"…"` keys would collide and React would warn and mix them up. Using `"start-ellipsis"` and `"end-ellipsis"` gives each a stable key.

Implementation steps

Q12. Why check `response.ok`?

`fetch` resolves on HTTP 404/500; it only rejects on network errors or abort. Without the check you would `slice` an error object (or HTML) and crash.

Q13. Why move focus to the heading after a page change?

If you click Next at the bottom of a long list, you stay scrolled down, and a screen reader announces nothing. Focusing the `<h1 tabIndex={-1}>` scrolls to the top and tells assistive tech where the new content starts; `aria-live` on "Showing…" announces the range.

Q14. How would you test the pagination?

Unit-test the pure helpers: `getPageRange` for every (current, total) combination up to 60 pages (fixed length, current included, ellipsis hides ≥ 2), `paginate`, `pageForNewSize` and the URL parsing. Use React Testing Library for Prev/Next disabled states, clicking a page, page-size change and error → Retry.

Performance and scale

Q15. The API returns 1 million rows. What changes?

Never load them all. Server-side pagination with a limit, the total from `X-Total-Count` (or `COUNT(*)`, maybe approximate), cursor pagination for very deep pages, and a cache of visited pages (React Query with `keepPreviousData` / `placeholderData`).

Q16. How would you make page changes feel instant with server-side pagination?

Prefetch the next page in the background, keep the previous page visible (faded) while loading, cache visited pages, and cancel the request for a page the user has already left (`AbortController`).

Q17. What if items are added or deleted while the user is paging?

With offset pagination, items can shift between pages (duplicates or skipped items). Use cursor-based pagination, or a stable snapshot / sort key (e.g. `createdAt + id`), and recompute and clamp the page when the total shrinks.

Trade-offs and "why not X?"

Q18. Why not `pushState` so the Back button goes to the previous page?

That's arguably better UX for pagination. It needs a `popstate` listener to read the page back from the URL, or a router (`useSearchParams`). I used `replaceState` for simplicity and would switch if the interviewer wants Back support.

Q19. Why not render all 500 items and let the user scroll?

500 simple cards would work, but DOM size grows with the data, the page gets slow on weak devices, and users lose their position. Pagination (or virtualisation) keeps the DOM small whatever the data size.

Q20. Why not a library like MUI Pagination or TanStack Table?

In production, yes: they're tested and accessible. In an interview, the point is to show the slice math, the ellipsis algorithm, edge cases and a11y. Then mention that you'd use a library.